

Block Building and Crafting Voxel System – Unity Asset

Developer Documentation

1. Installation

You can purchase the **Block Building and Crafting Voxel System** package from the Unity Asset Store and import it via **Package Manager**. After importing, you'll see a new folder called:

`Assets/BlockBuildingCraftingSystem/`

Open the **Scenes** folder and play the included **GamePlay Demo Scene** to explore all systems in action.

All gameplay elements are prefab-based and modular — you can easily reuse or customize them for your own voxel or survival-style games.

If you have any questions, please contact me at queendev95@gmail.com 😊

2. Core Prefabs

All prefabs are organized under:

`Assets/BlockBuildingCraftingSystem/Prefabs/`

You can drag and drop them directly into your scene or copy them from the GamePlay scene. We have got two sample scenes. These are Menu and GamePlay.

2.1 Player Prefab

Prefab: **BlockBuildingCraftingManager** and its child: **Character**

This prefab contains all player and main game systems:

BlockBuildingCraftingManager:

- **BlockBuildingCraftingManager** – you decide the controller type here. It manages general things.
- **Weather Manager** – manages weather.
- **DayNightManager** – manages the day - night cycle.
- **Inventory Manager** – manages the general inventory items.
- **Save Manager** - saves and loads the inventory and current map.
- **Block Database** - contains all the blocks and items.
- **World Initializer** - initializes and loads the world map.
- **Crafting Manager** - contains and manages all recipes, crafting panel, etc.

Character:

- **FirstPersonController** – handles movement, gravity, sprint, and jump.
- **HeroPlayerScript** – manages health, stamina, regeneration, and player state.
- **FPSHandRotator** – animates held tools and switches between hotbar items.
- **Camera** – attached to the player for first-person control.

You can switch between **PC** and **Mobile** controls via `BlockBuildingCraftingManager → controllerType`.

3. Block and World System

3.1 Voxel World Generator

The **VoxelWorldGenerator.cs** script procedurally creates voxel terrain based on probabilities:

- Earth, Stone, Iron, Bedrock, and Bush blocks.
- Simple random generation logic for trees and ores.
- Chunk-based structure (`GenerateChunk` method).

💡 Tip: You can generate your own world by calling
`VoxelWorldGenerator.GenerateChunk(parentTransform, chunkSize)`

3.2 Block Database

All available blocks are listed inside the **BlockDatabase** component. Each block entry (`BlockDataEntry`) contains:

- `id` → unique block identifier (e.g. "stone", "wood")
- `prefab` → prefab of the block or item
- `icon` → UI sprite for inventory/crafting

- `type` → Placeable or Item
 - `isCraftable` → If the block can be crafted
-

4. Block Mechanics

4.1 BlockInteraction

Handles **breaking** and **placing** blocks:

- **Left Click (Mouse 0)**: Break the targeted block.
- **Right Click (Mouse 1)**: Place the selected block from hotbar.
- Works with both keyboard/mouse and mobile input.

Important:

- Blocks must have `BlockHealth.cs` attached.
- Placement requires having the item in inventory.

4.2 BlockHealth

Each block has health and break logic:

- `maxHealth` → how many hits are required to destroy it.
- When destroyed, plays sound, particles, and spawns loot.
- Calls **SaveManager** to mark destroyed blocks.

Optional FX: You can assign break particles, hit colors, and loot prefabs.

4.3 BlockLootItem

Floating collectible items that rotate and move toward the player.

When near the player:

- Adds the item to **InventoryManager**.
 - Plays pickup sound and disappears.
-

5. Inventory and Hotbar

5.1 InventoryManager

Handles player's items and saving system:

- Each item has `id` and `quantity`.
- Supports scrolling with mouse wheel (or mobile UI buttons).
- Automatically saves to `inventory_save.json`.

Key Methods:

```
AddItem(string id, int amount);  
RemoveItem(string id, int amount);  
SaveInventory();  
LoadInventory();
```

5.2 UI_Hotbar

Displays player's current items visually.

Automatically highlights selected slot and updates icons based on **BlockDatabase**.

6. Crafting System

6.1 CraftingManager

Controls the in-game crafting UI and logic.

Open and close it using **C key** (PC) or **Crafting Button** (mobile).

Features:

- Displays all available recipes.
- Checks materials from the player's inventory.
- Automatically adds crafted items to inventory.

Recipe Definition:

Use `CraftingRecipeSO` ScriptableObject.

Each recipe includes:

- Output item (ID, Icon, Amount)

- Ingredients list (IDs and required amounts)

Location:

`Assets/BlockBuildingCraftingSystem/ScriptableObjects/`

6.2 UI_CraftingButton

Each button in the crafting panel triggers crafting:

```
CraftingManager.Instance.CraftItem(recipe);
```

7. World Saving System

7.1 SaveManager

All block positions are serialized to:

`Application.persistentDataPath/world.json`

Saved data includes:

- Placed blocks (IDs and positions)
- Destroyed blocks (to prevent regeneration)

Automatic saving every 10 seconds.

Supports:

```
SaveWorld();  
LoadWorld();  
HasSavedWorld();
```

7.2 WorldInitializer

When loading the game, it restores all previously placed blocks using data from `world.json`.

Also starts an **auto-save coroutine** to continuously back up your world.

8. Environmental Systems

8.1 Day & Night Manager

Simulates real-time light cycle:

- Adjustable sunrise and sunset hours.
- Smooth ambient color transition.
- Toggles all light sources at night automatically.
- Events: `EventToInvokeWhenSunRise`, `EventToInvokeWhenSunSet`.

Setup:

Attach your **Directional Light** and **Skybox Material** to the manager.

8.2 Weather Manager

Randomly triggers **rain** or **clear sky** weather effects using a particle system.

You can adjust `weatherChangeInterval` for shorter or longer cycles.

9. Audio & Game Canvas

AudioManager

Handles:

- Jump sounds
- Button clicks
- Ambience (loopable background)

Access from anywhere:

```
AudioManager.Instance.Play_Button_Click();
```

GameCanvas

Manages all UI panels:

- Pause / Settings / Map / Crafting
- Updates health and stamina bars
- Switches between mobile & PC interface automatically

10. Gameplay Controller

BlockBuildingCraftingManager

The central game controller:

- Holds `PlayerValues`
 - Defines `ControllerType` (PC / Mobile)
 - Initializes camera, HUD, and frame rate
 - Syncs health and stamina between player and UI
-

11. Useful Tips

✓ Creating new blocks:

1. Duplicate an existing prefab.
2. Change mesh, collider, and material.
3. Add `BlockHealth.cs` and configure `blockID`.
4. Add it to **BlockDatabase** list.
5. Done! You can now place, break, and craft it.

✓ Creating new recipes:

- Right-click in Project window →
 `Create` → `Crafting` → `Recipe`
- Fill ingredients and output fields.

✓ Mobile Ready:

Joystick, buttons, and touchpads appear automatically when you select `ControllerType.Mobile`

12. How to create your own Block

Creating your own custom block is **extremely simple** with the Block Building and Crafting Voxel System.

Just follow these steps carefully:

Step 1 – Create Your Block Prefab



1. In your Project window, right-click → **Create** → **3D Object** → **Cube** (or import your own mesh). OR YOU CAN SIMPLY DUPLICATE ONE OF THOSE IN THE Prefabs\Blocks.
2. Rename it to something meaningful like **Stone_Block** or **Iron_Block**.
3. Add a **Box Collider** if it doesn't have one.
4. Make sure the object has a **Renderer** component (so the crack visual updates properly).

Step 2 – Add Required Components

Attach the following scripts:

- **BlockHealth.cs**
This controls the block's durability, destruction FX, and loot behavior.
Configure in the Inspector:
 - **blockID** → must be a **unique string** (e.g. "stone", "iron")
 - **maxHealth** → how many hits before breaking
 - **breakSound** → optional audio clip
 - **breakParticlesPrefab** → particle effect on break
 - **lootItemPrefab** → optional drop item

Example:

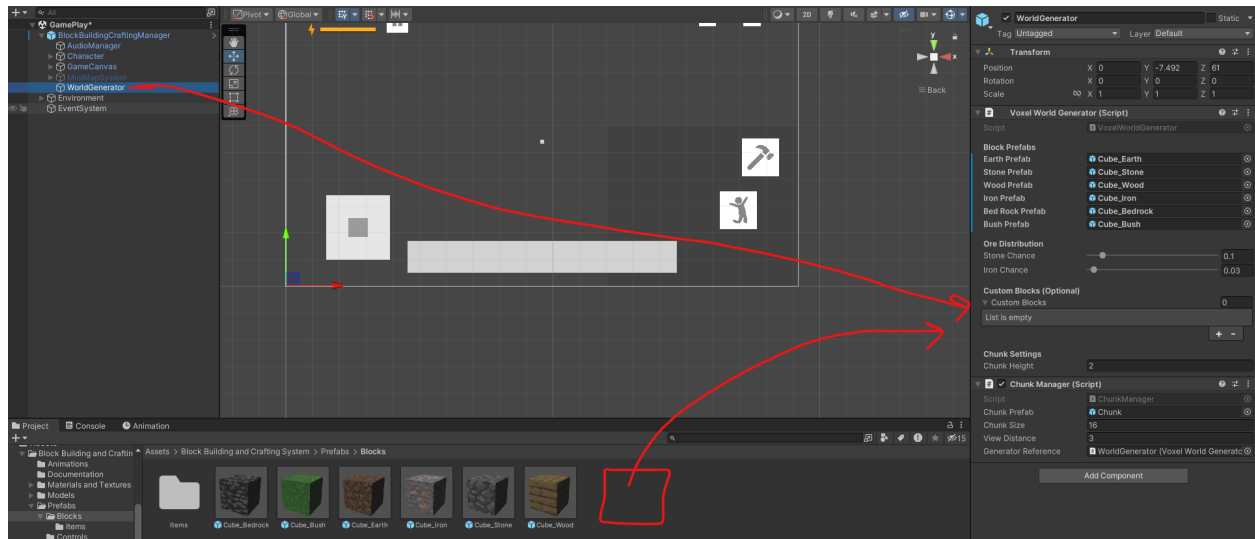
```
blockID: "stone"  
maxHealth: 5  
lootItemPrefab: Dirt_Loot
```

Step 3 – Register Your Block in BlockDatabase

1. Locate the **BlockDatabase** object in your scene.
2. In the **blocks** list, click **+** to add a new entry.
3. Fill these fields:
 - **ID:** Must match your block's **blockID**
 - **Prefab:** Drag and drop your block prefab
 - **Icon:** Sprite to display in Hotbar / Crafting UI
 - **Type:** Select **Placeable**

- **isCraftable:** `false` (unless you want it craftable later)

🔨 Step 4 – Add to WorldGenerator\VoxelWorldGenerator Component



Now that you have created your new block, it's time to make it naturally appear in the world.

The **VoxelWorldGenerator** component is responsible for creating the terrain and distributing all blocks across the map.

It already generates basic layers such as **Earth**, **Stone**, **Iron**, **Wood**, **Bush**, and **Bedrock** — but it also allows you to **inject your own custom blocks** into the world with full control over spawn chances.

🧭 a. Open the World Generator

In your **Scene Hierarchy**, select the object that has the **VoxelWorldGenerator** component (usually attached to a “World” or “Generator” GameObject).

🧱 b. Add Your Custom Block

Under **Custom Blocks (Optional)** you will find a list named “**Custom Blocks**”.

Click the “+” button to add a new element and assign. 💡 *Tip:* You can add as many custom blocks as you like — each one will be randomly mixed into the world generation alongside the base terrain blocks.

🌍 c. Generate the World

Once you've added your new block(s):

1. Press **Play** to start the game.

2. The **VoxelWorldGenerator** will automatically:

- Build chunks layer by layer
- Insert your **custom blocks** at random positions according to their chance value
- Keep **Bedrock** at the bottom layer
- Spawn **trees and bushes** naturally on the top surface

If you don't see your block, increase its **Chance** value and re-generate the world (delete old **world.json** first to force a clean generation).

⚙️ d. Save & Reload

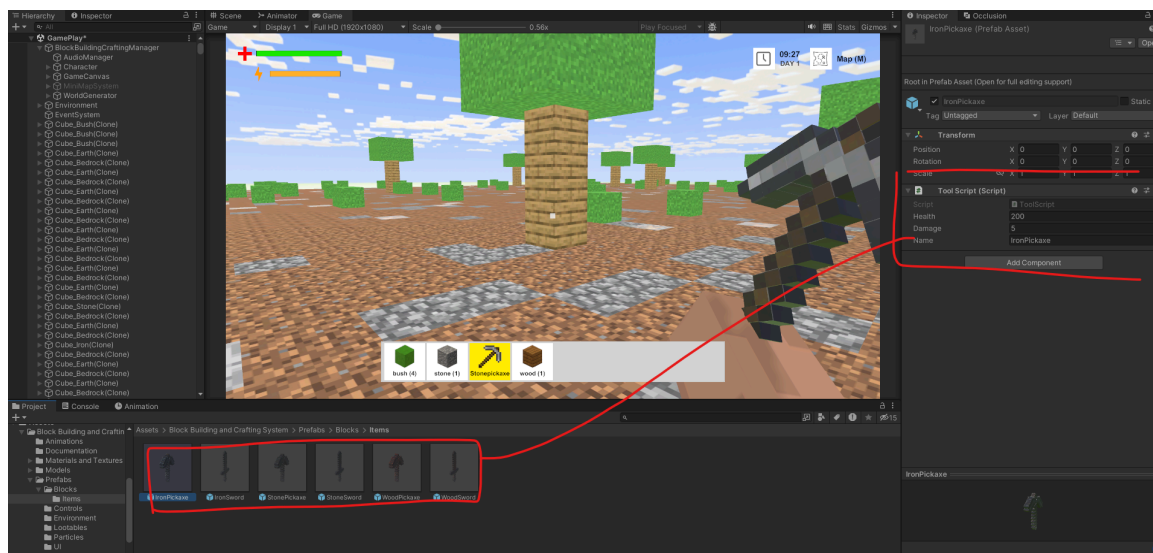
All generated blocks, including your custom ones, are automatically saved into **world.json** by the **SaveManager**.

When you reload the game, they'll appear in exactly the same positions.

Run the **GamePlay** Scene and use **Right Click** to place your new block, or **Left Click** to break. When broken, it should:

- Play break sound
- Show particle FX
- Drop the loot item (if assigned)
- Be correctly saved and reloaded by **SaveManager**

13. How to create your own Craftable Item

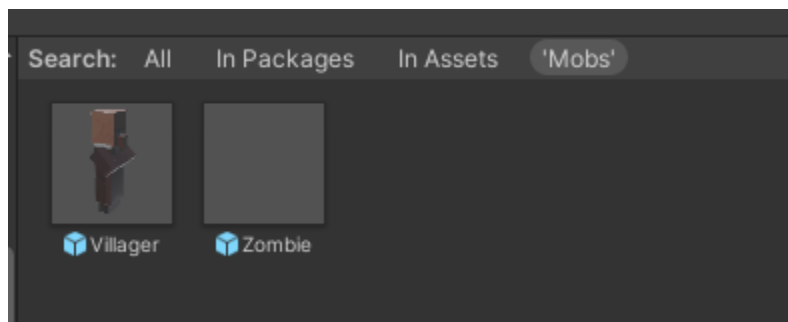


The crafting system allows you to easily create new **tools**, **items**, or **blocks** that players can craft in-game using materials they've collected.

You can create any kind of craftable object without writing a single line of code — everything works through **ScriptableObjects** and the **CraftingManager**.

14. Mob AI System

Two new mob types have been added to the package: Enemy Mob and Neutral Mob. The system is fully prefab-based and modular. You can easily integrate your own custom mobs by duplicating an existing mob prefab and replacing its body mesh model, adjusting colliders, and tuning all behavior parameters inside the Mob AI scripts.

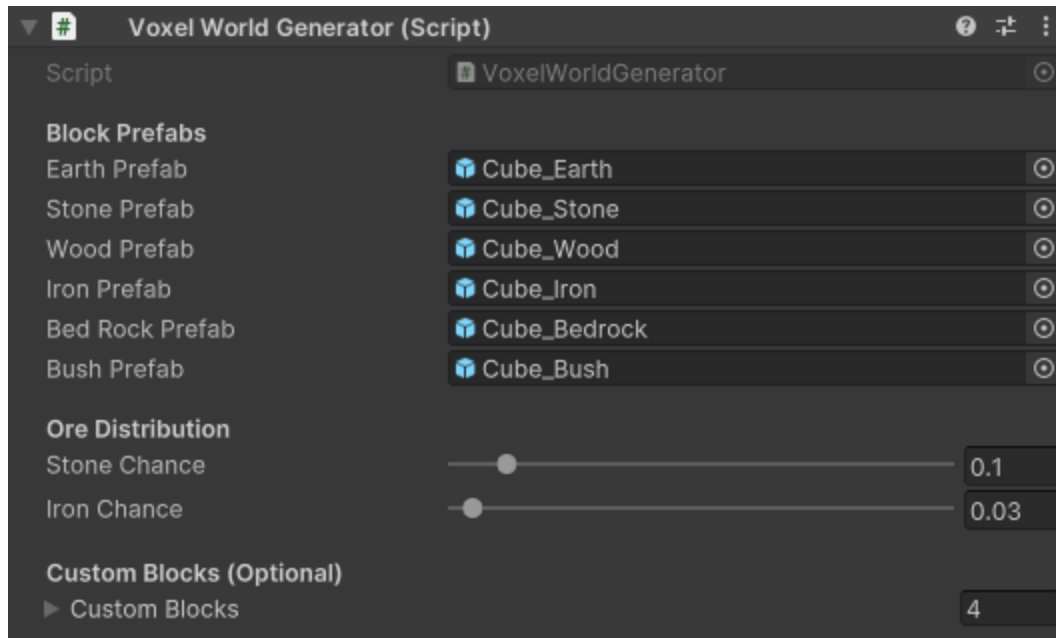


This allows you to configure aggression, speed, detection range, interaction type, and more — without writing any code. The Mob AI system works seamlessly with the chunk-based world structure and the SaveManager persistence flow, ensuring that mob interactions do not break the world save/load cycle.

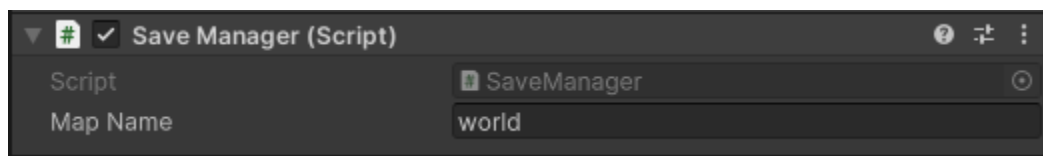
15. Biome Sample Scenes

Three new biome scenes have been included in the package: Forest, Desert, and Snow. When you play these scenes, you will notice that biome-specific blocks are automatically generated for each environment.

These blocks are stored in separate JSON files per biome, because the VoxelWorldGenerator has predefined block assignments for building each biome layer, and the SaveManager saves the world independently based on the Map Name.



This architecture ensures biome-based world generation remains consistent, saved data stays organized, and new biome block sets can be expanded easily. All biome scenes are fully compatible with chunk generation and world persistence using world.json.



16. Gamepad Support

Partial Gamepad support has been added to all gameplay input mechanics — excluding the main menu UI. You can now control: character movement (walk, sprint, jump), hotbar slot switching, block placement, and block breaking using a Gamepad.

The input mode automatically adapts based on BlockBuildingCraftingManager → controllerType, supporting both PC and Mobile builds with native Gamepad detection and integration.

Step 1 – Create a New Recipe Asset

In the **Project Window**, right-click anywhere and select:

Create → Crafting → Recipe

This will create a new asset of type **CraftingRecipeSO**.

You can rename it to something descriptive — for example, `Recipe_Pickaxe` or `Recipe_IronBlock`.

Step 2 – Configure the Recipe

Select your newly created asset and fill in the following fields in the Inspector:

Field	Description
Output ID	The ID of the item or block that will be produced when crafted. <i>(This must match the ID from your BlockDatabase.)</i>
Output Amount	How many items will be created. <i>(Example: 1)</i>
Icon	The sprite shown in the crafting menu for this recipe.
Ingredients	A list of required materials for crafting. Each ingredient has: → id : Item ID (must exist in BlockDatabase) → amount : Required quantity of that item.

Example:

To craft a **Wooden Pickaxe**, you could define:

Ingredient ID	Amount
wood	3
stone	2

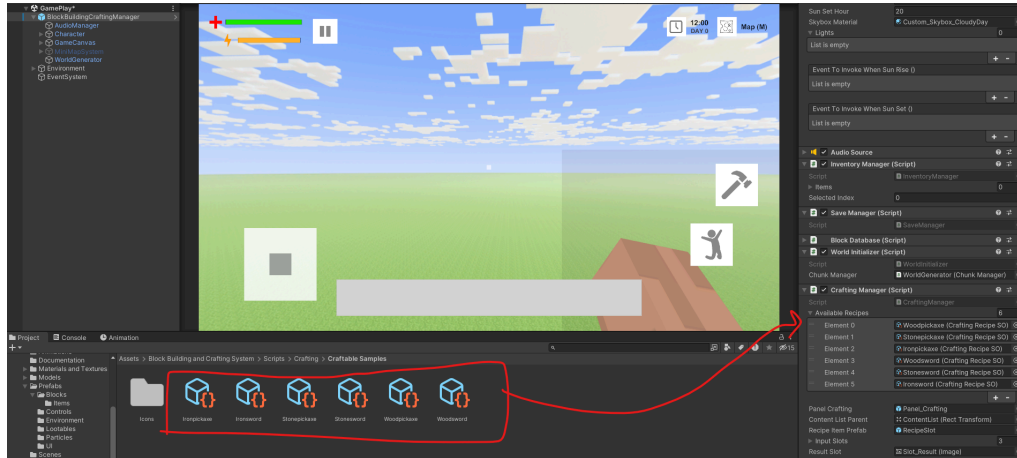
And set **Output ID** = "pickaxe_wood" and **Output Amount** = 1.

Step 3 – Add Your Recipe to the Crafting Manager

Now that your recipe asset is ready:

1. In your **Scene Hierarchy**, find the object with the `CraftingManager` component. In the Inspector, open the **Available Recipes** list.

- Click the “+” button and drag your new **CraftingRecipeSO** asset into the new slot.



Your recipe will now automatically appear inside the in-game Crafting Panel.

Step 4 – Open the Crafting Panel and Test

During gameplay:

- On **PC**, press **C** to open the crafting window.
- On **Mobile**, tap the **Crafting Button** on the screen.

All available recipes will appear with their **icons and names**.

When you select one:

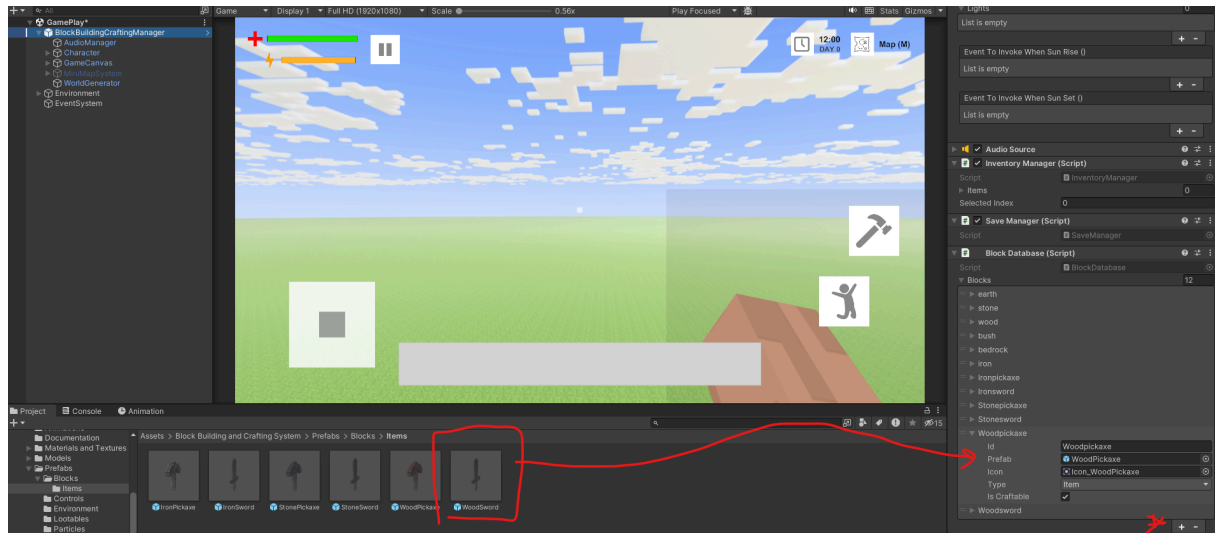
- The system checks if the player has the required ingredients in the inventory.
- If yes — ingredients are removed, and the crafted item is added to the player's inventory.
- If not — a warning message is shown in the Console (**Not enough materials to craft: [itemID]**).

Step 5 – Make the Crafted Item Functional

After crafting, your new item will behave depending on how it's defined in the **BlockDatabase**:

Placeable Can be placed in the world like any other block (e.g., walls, furniture, decorative blocks).

Item Appears as a usable tool in the player's hand via the **FPSHandRotator** (e.g., Pickaxe, Axe, Hammer, etc.).



To control this behavior:

1. Open your **BlockDatabase** in the scene.
2. Add your new item prefab with a **unique ID** (matching the Output ID).
3. Set the **Type** to either:
 - **Placeable** → for blocks
 - **Item** → for handheld tools
4. Assign the prefab and its icon.

Now, your new crafted object will integrate seamlessly with:

- The **Hotbar UI**
- The **InventoryManager**
- The **BlockInteraction** system
- The **Save/Load** persistence

🧠 Optional: Advanced Uses

✅ You can combine crafting with your **custom blocks** — for example:

- Add a “Diamond Block” to the BlockDatabase.
- Then create a recipe that crafts it from 9 diamond ores.

✅ Recipes can include **any item**, not just blocks — you can even craft a “Tool”, “Potion”, or “Energy Cell”.

✅ To make your crafting system expandable, you can prepare **multiple CraftingRecipeSO** assets and group them (e.g., tools, furniture, materials).

14. Contact

If you face any issue or want to suggest new features, please reach me anytime:

 queendevolver95@gmail.com

Kind Regards,

Queen Developer

Creator of *Block Building and Crafting Voxel System*  